SHOP EASY - PROJECT REFLECTION REPORT

Azra İrem Dalhasanoğlu - 220717049

1. Bugs Found in the Starter Code by Testing Techniques

While working on the test suite for the ShopEasy e-commerce library, I used different testing methods, and they helped me find some interesting bugs and problems in the starter code.

First, when I was doing Specification-Based Testing (Chapter 2) and Design by Contract (Chapter 4), I noticed that the code did not check the inputs carefully. For example, if a user tries to add a product with a zero or negative quantity using the `ShoppingCart.addItem()` method, the starter code did not have any checks for this. It just accepted it, which can break the sepet logic later. By adding Java `assert` statements for the preconditions, I managed to stop these bad inputs right at the beginning of the methods.

The biggest and most surprising bug was found during Property-Based Testing (Chapter 5) using `jqwik`. I was testing the Cart Commutativity property, which basically means that if you add a list of items to the cart in normal order or backwards order, the final price must be exactly the same. However, `jqwik` found a failing case. It showed that changing the order of items caused small differences in the total price, like getting `3.0` instead of `3.12`, or having very tiny decimal differences.

The root cause of this bug is the Floating-Point Rounding Error in Java's `double` type. Computers save floating-point numbers in binary format, so they cannot represent every decimal number perfectly. When the shopping cart loops through items and adds prices together, these tiny errors accumulate. Since the order of addition changes when we go backwards, the final rounded number changes too. This makes the strict `isEqualTo` check fail. I fixed this by changing the test to use `isCloseTo(..., within(0.01))` instead of checking for exact equality.

2. Test Effectiveness per Unit of Effort

In my opinion, Property-Based Testing (PBT) was the most effective technique considering the effort it requires. Writing a property test with `jqwik` takes almost the same time and code as writing two or three regular unit tests. But the result you get is much bigger.

With normal unit tests, you just type some manual numbers and test them. But with PBT, you just write one general rule and create a data provider using `@Provide`. After

that, the framework automatically creates and runs thousands of random test cases, including very strange edge cases that a human would never think of. Finding that floating-point accumulation bug by writing manual tests would take days and hundreds of lines of code. `jqwik` found it in seconds. This makes it super efficient for developers.

3. Test Suite Classification nd Missing Elements

Looking at the Testing Pyramid from Chapter 1, my test suite is 100% a Unit Testing Suite. All the tasks I did—like checking boundaries in `PriceCalculator`, checking branches in `ShoppingCart`, and using mocks for `OrderProcessor`—run completely in the computer's memory. They do not talk to any real database or external network.

To make this a real, production-ready system, we are missing Integration Tests and End-to-End Tests. Mocking with Mockito in Task 5 was great because it allowed me to test the logic of `OrderProcessor` easily. However, in real life, we don't know if the system can actually talk to a real PostgreSQL database or a real payment API. We are missing tests that check if database queries, configuration files, and network responses work together without mocks.

4. What to Test Next (With One More Week)

If I had one more week for this project, I would focus on much simpler and practical improvements. First, I would write basic unit tests for the missing getter, setter, and constructor methods in the data classes like `Product` and `Order`. Right now, our test suite mostly focuses on complex logic like `ShoppingCart` and `PriceCalculator`, but we should make sure the simpler classes are also completely tested.

Secondly, I would spend more time on PIT mutation testing. In Task 2, we saw that it creates mutants to check our test quality. With one extra week, I would run the mutation coverage report again and add a few more simple test cases to kill any surviving mutants. This would make our current unit tests much stronger without adding too much complex code to the project.